

KT그룹

AX 전략기획 및 멀티에이전트 서비스 개발

Agentic AI AI-Native 개발 방법론



CONTENTS

목차

01 AI와 협업하는 개발

02 업의 분석과 워크플로우 설계

03 컨텍스트 엔지니어링과 구조화 출력

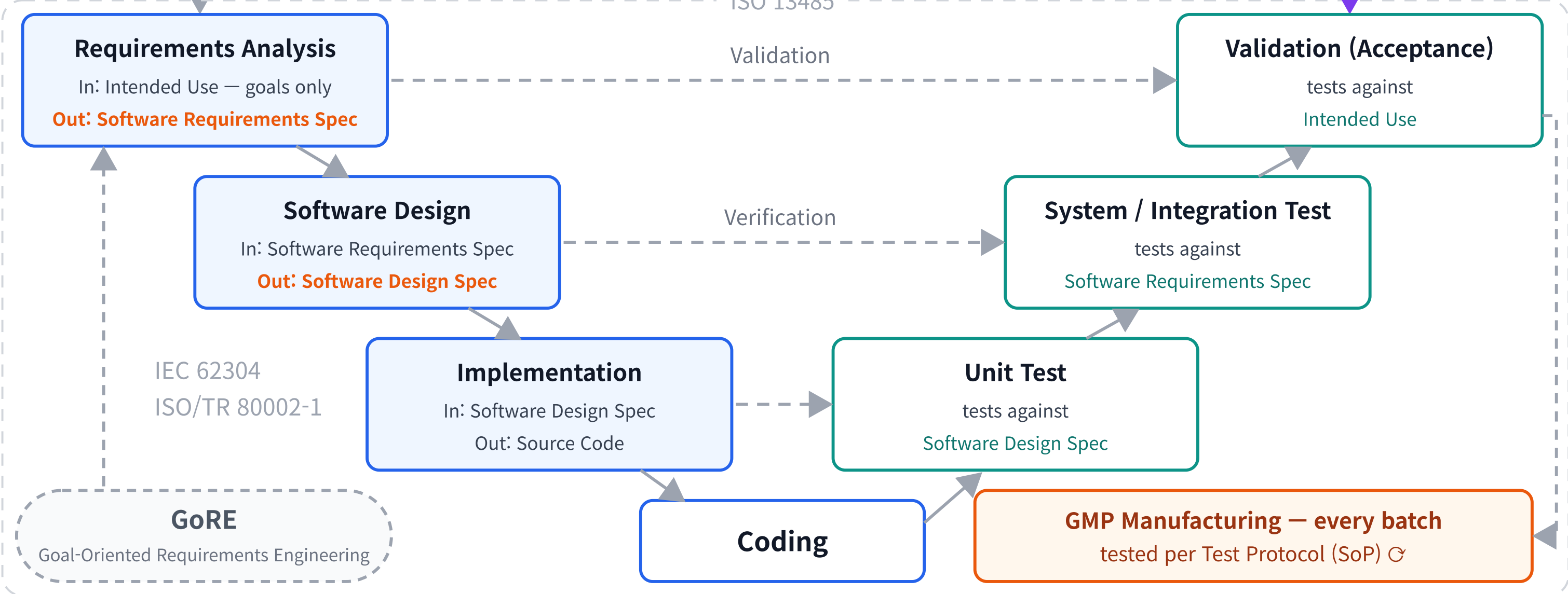
과정 전체 지도



일을 제대로 했다면, 형식과 관계없이
나와야만 하는 산출물(문서)이 있습니다.

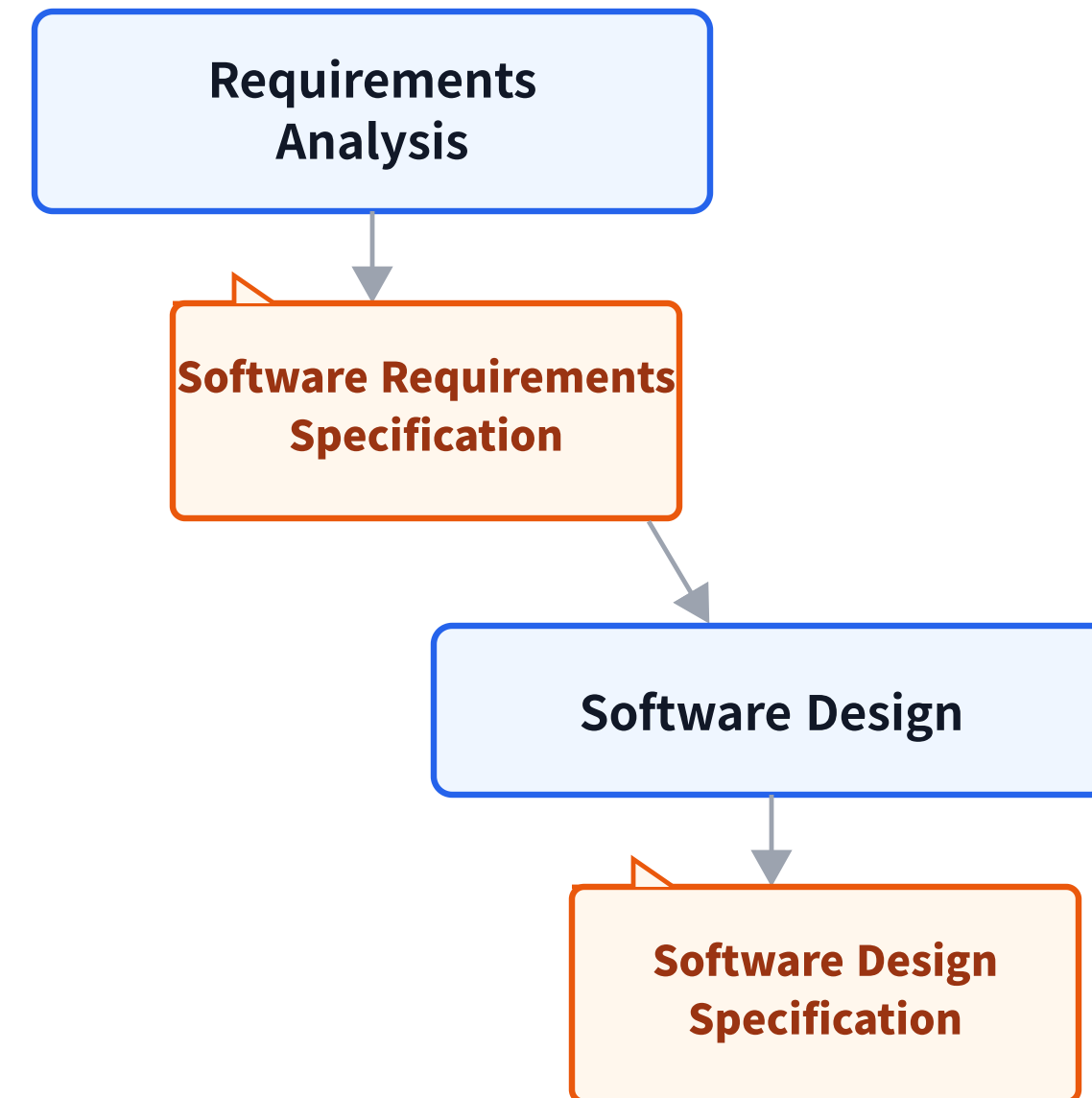
단순 행정적인 잡무가 아닙니다.

1



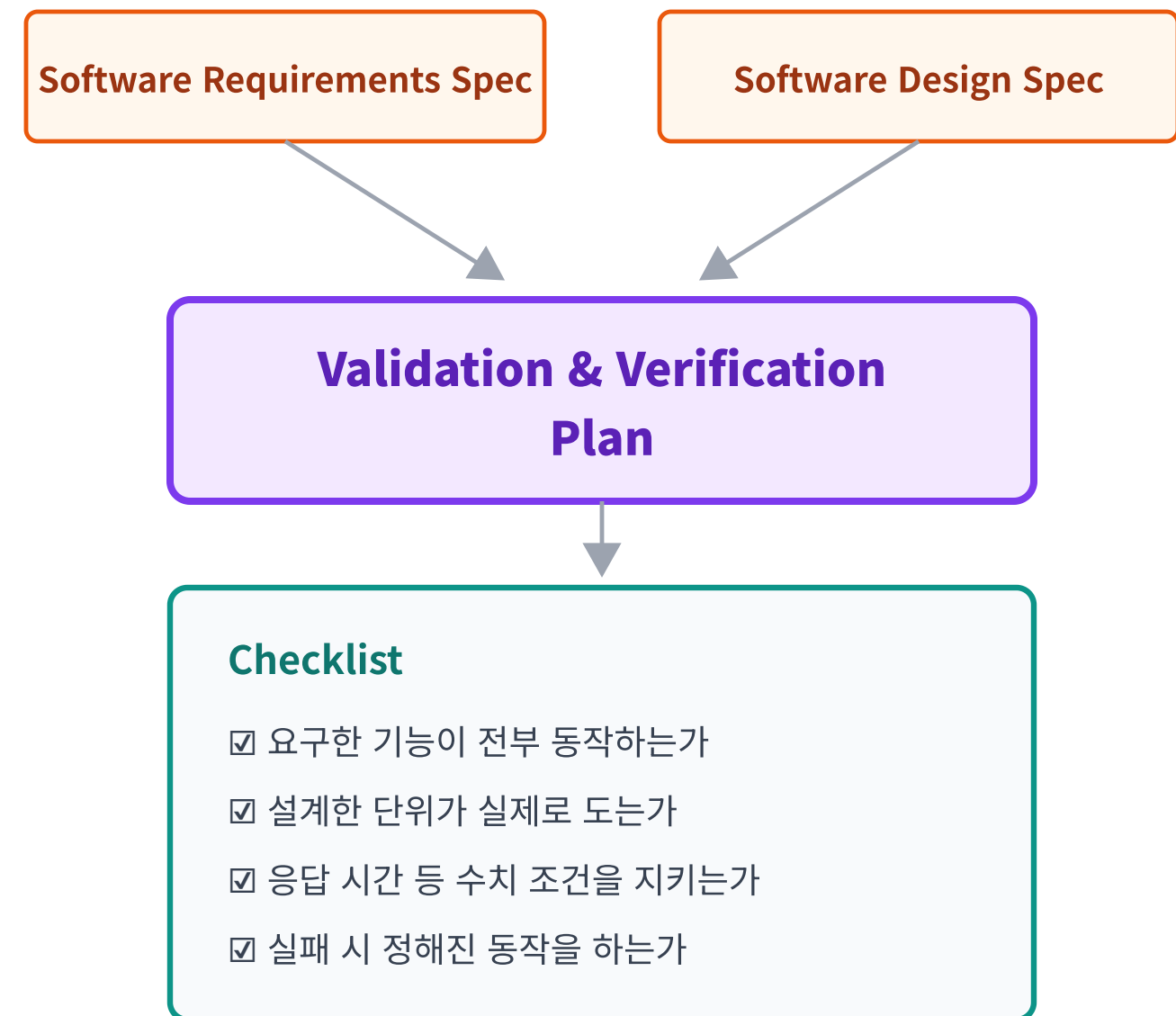
요구사항 분석과 설계

- **요구사항 분석**: 문제와 비즈니스 차원의 요구를 소프트웨어에 요구되는 사항으로 정리하는 과정
- 핵심 질문 = 목표 달성에 이 요구사항이 필요한가
- 요구사항 분석 → Software Requirements Specification
소프트웨어 설계 → Software Design Specification
- **Spec**-Driven Development(SDD): 스펙 문서를 먼저 작성하고 AI와 함께 개발하는 방식



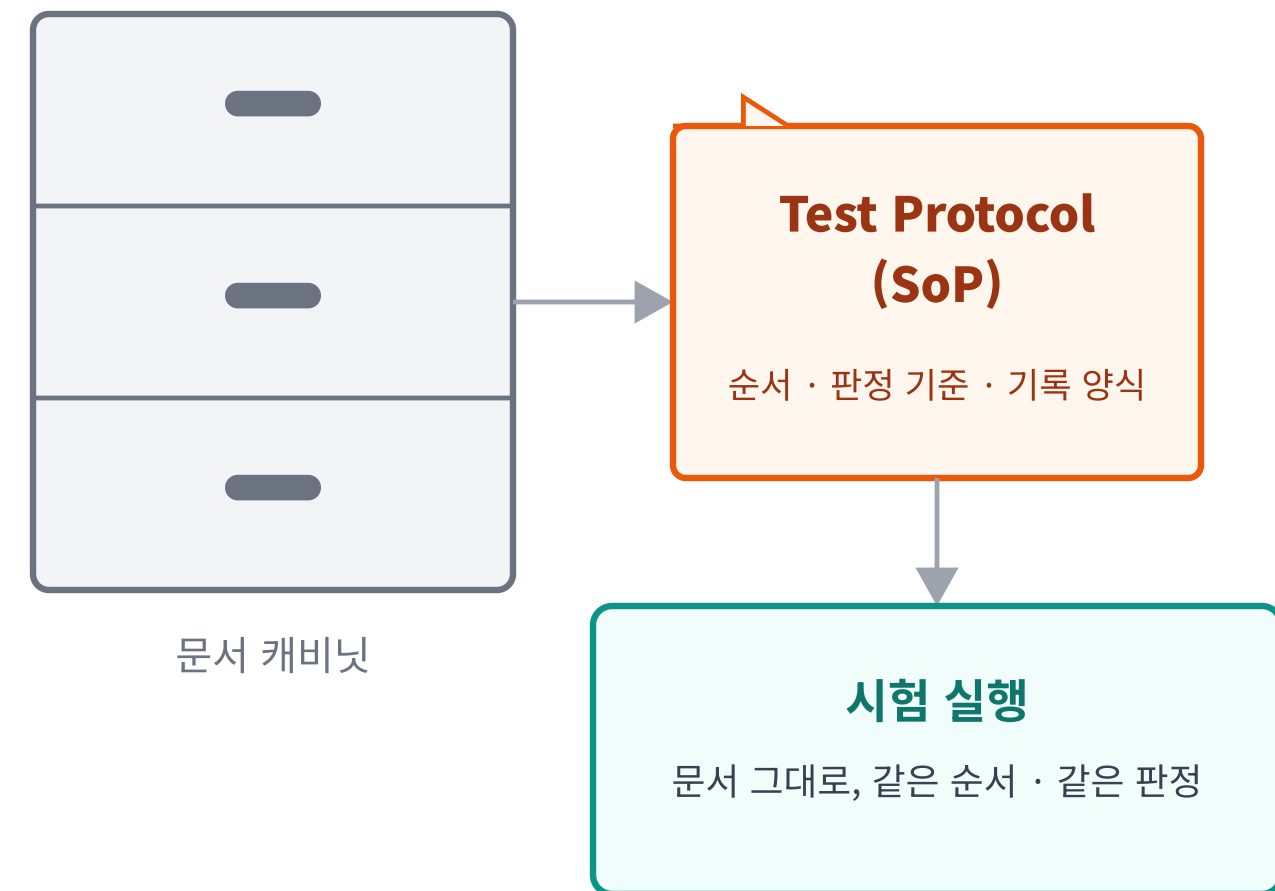
Validation & Verification Plan

- 두 명세가 서면, 그 뒤를 이어 **Validation & Verification Plan**을 세웁니다.
- 모든 요구사항과 설계사항, 각 단위별 코드 구현물들에 대해 통과해야 할 **Checklist**가 나옵니다.
- **Test-Driven Development(TDD)**: 통과할 검사를 먼저 정해 두고, 그 검사를 통과하도록 구현하는 개발 방법
- 달성할 목표가 명확하면, 에이전트가 **루프**를 돌며 완성할 때까지 작업을 시킬 수 있습니다. 통과 조건이 곧 종료 조건입니다.



표준작업절차서

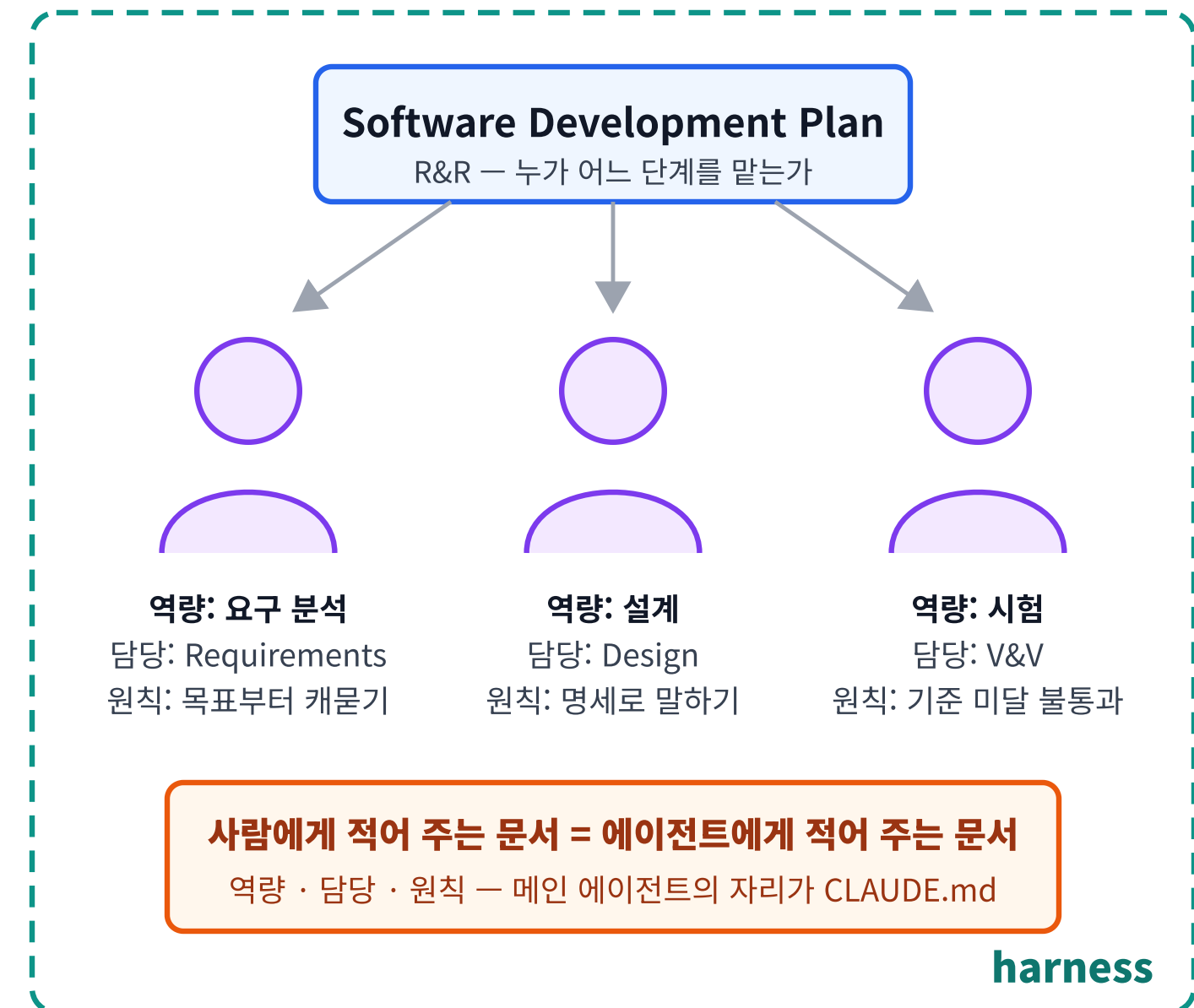
- 작업 절차가 수립되면 제조/배포할 때마다 정해진 프로토콜에 따라 매번 똑같이 검사를 합니다.
- 누가 작업을 하더라도 똑같이 수행할 수 있도록 **표준작업절차서(SoP)**를 만들어야 합니다.
- **SoP(Standard Operating Procedure):** 무슨 작업을 어떤 순서로 어떻게 해야 하는지 상세히 적어 둔 문서
- **Skill:** 에이전트가 무슨 작업을 어떤 순서로 어떻게 해야 하는지 적어 둔 문서



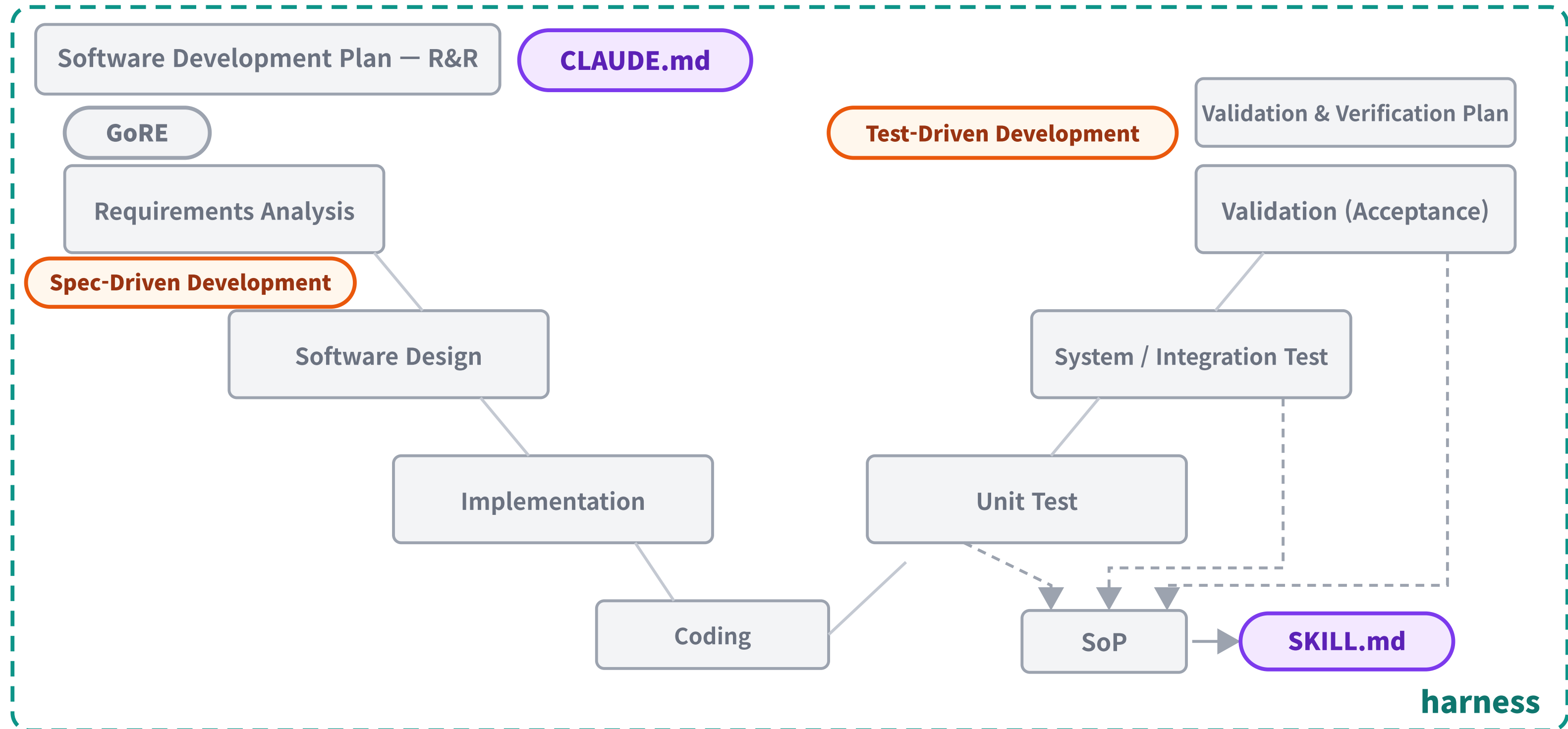
제조 회차마다 같은 문서로 반복 ♻

개발 계획의 R&R

- 개발 계획을 세우면서 **R&R**을 정합니다.
어떤 역량을 가진 사람이, 왜 어떤 분야를 맡아,
어떤 원칙과 철학으로 일해 주기를 바라는지
정의합니다.
- **에이전트 문서**: 어떤 역량으로 어떤 분야를 맡아
어떤 원칙으로 일하는지를, 에이전트에 대해 적어
둔 문서
- **CLAUDE.md**: 클로드 코드의 메인 에이전트의
역할과 원칙을 적는 문서
(일반형 표기는 AGENTS.md)
- **harness**: 에이전트가 일하는 방식과 절차 전반을
정해 주는 실행 환경



에이전트 세상, Classical Modern

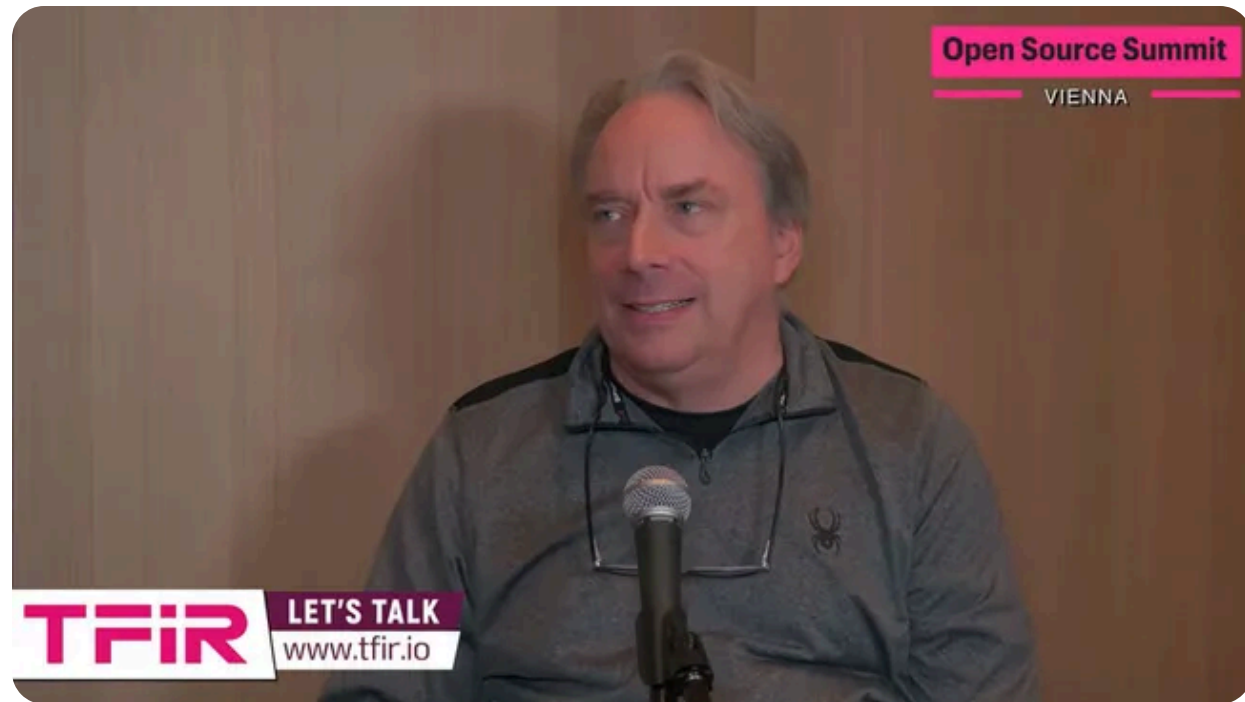


Linus Torvalds



- **Linux 커널** 최초 공개, 1991년.
지금 세계의 서버와 클라우드, 안드로이드 단말이 이 커널 위에서 돕니다.
- **Git** 개발, 2005년.
커널 개발을 감당할 버전 관리 도구를 직접 만들었고, 지금은 사실상 모든 개발자가 이것을 씁니다.
- 지금도 리눅스 커널의 최종 관리자입니다.

2024년 10월, Open Source Summit Europe



“I think the whole tech industry around AI is in a very bad position and it's 90 percent marketing and ten percent reality.”

AI를 둘러싼 기술 업계 전체가 아주 나쁜 상황에 있다고 봅니다. 90퍼센트가 마케팅이고 10퍼센트가 현실입니다.

2025년, 온라인에 돌던 대화 한 장



Very Inefficient But Entertaining

합성된 가짜 mim입니다.

- 빌 게이츠와 리누스 토발즈가 vibe coding을 두고 주고받았다는 이미지로 돌았습니다.
- 두 사람의 실제 게시물은 확인되지 않습니다.

2026년 1월, AudioNoise README

```
# AudioNoise
```

Note that the python visualizer tool has been basically **written by vibe-coding**, and I did not do any of the tweaking myself.

I cut out the middle-man (me) and just used **Google Antigravity** to do the audio sample visualizer.

- 리눅스 커널과 Git을 만든 사람(리누스 토발즈)이, 자기 취미 프로젝트의 도구를 vibe coding으로 만들었다고 **스스로 적어 둔 문서**입니다.
- 사용한 도구는 **Google Antigravity**.

2026년 7월, 리눅스 커널 메일링 리스트



lore.kernel.org LKML 원문 (2026. 7. 14.) · phoronix.com 전문 인용 / 2026-08-22 확인
AI-Native 개발 방법론 / 개요 / **Linus Torvalds**

Linux is not one of those anti-AI projects, and if somebody has issues with that, they can do the open-source thing and fork it. Or just walk away.

리눅스는 반AI 프로젝트가 아닙니다. 그게 문제라면 오픈소스답게 포크하거나, 그냥 떠나면 됩니다.

'is it useful' is no longer one of those questions. Anybody who doubts that clearly hasn't actually used it.

'쓸모가 있느냐'는 이제 물을 필요가 없는 질문입니다. 이를 의심하는 사람은 실제로 써 보지 않았을 겁니다.

AI is a tool, just like other tools we use. And it's clearly a useful one. It may not have been that 'clearly' even just a year ago, but it's no longer in question today.

AI는 우리가 쓰는 다른 도구와 같은 도구이고, 분명히 쓸모 있는 도구입니다. 1년 전만 해도 그렇게 '분명'하지는 않았지만, 지금은 논쟁 대상이 아닙니다.

AI-Native 개발 방법론

AI와 협업하는 개발

역할의 재편 / 생명주기 / ReAct 읽기

학습 목표

- 사람 몫(무엇을 · 왜 · 판단)과 AI 몫(구현 · 반복)이 왜 갈리는지 설명할 수 있습니다.
- 기존의 DLC(Development Life Cycle)과 AI-DLC의 차이점을 설명할 수 있습니다.
- AI 에이전트의 문제 해결 흐름을 설명할 수 있습니다.

▶ AI와 협업하는 개발

업의 분석과 워크플로우 설계

컨텍스트 엔지니어링과 구조화 출력

신입 개발자 채용 시장의 변화

줄어드는 자리

"신입 개발자 고용, 약 20% 감소"

미국 급여 데이터 실측 연구에서, 22~25세 소프트웨어 개발자 고용이 2022년 말 이후 약 20% 줄었습니다.

남는 자리

"AI가 대체하지 않는 일"

같은 연구에서, AI가 사람을 대체하기보다 보조하는 쪽으로 쓰이는 직무에서는 같은 연령대의 고용 감소가 나타나지 않았습니다. 일 자체가 없어진 것이 아니라, **일의 모양이 바뀌었습니다.**

줄어든 것은 개발자가 아니라, 코드만 치는 자리입니다.

AI 도입 이후의 코드 품질 지표

지표 (2.11억 줄 분석)	2020	최근
복사 · 붙여넣기 줄 비율	8.3%	15.7%
리팩토링(움겨 정리한) 줄 비율	24.1%	9.5%
중복 코드 블록	기준	약 10배

- AI가 코드를 쏟아내면서, 복사해 붙이는 일은 두 배 가까이 늘고, 움겨 정리하는 일은 절반 이하로 줄었습니다.
- 쏟아진 코드는 **검증하고 정리해야 할 빚**, 기술 부채(Technical Debt)로 쌓입니다. 그 빚을 다루는 사람이 필요합니다.

개발자 역할의 재정의

The programmer is increasingly not just a code writer, but an orchestrator of agents.

프로그래머는 점점 코드를 쓰는 사람이 아니라 에이전트를 지휘하는 사람이 되어 갑니다.

Right now the agents are like interns. You still have to be in charge of aesthetics, judgment, taste, and oversight.

지금 에이전트는 인턴에 가깝습니다. 안목과 판단, 감독은 여전히 사람이 맡아야 합니다.

You can outsource your thinking, but you can't outsource your understanding.

생각은 외주를 줄 수 있어도, 이해는 외주를 줄 수 없습니다.

- 코드를 **쓰는(Write)** 사람에서, 쏟아진 코드를 **읽고 검증하고 판단하는** 사람으로 자리가 옮겨 갑니다.

**더욱 더 넓은 시야를 갖추고
일해야 하는 사람이어야 합니다.**

AI와의 개발은 코드를 치는 일에서,
일을 분해해 체계(system)를 잡는 일로 바뀝니다.

이렇게 바뀐 개발의 흐름 전체를 AI-DLC(AI-driven Development Life Cycle)라고 부릅니다.

개발자 산출물의 이동

이전

개발자 → 직접 코딩 → 코드

사람이 코드를 타이핑했습니다.
코드가 곧 결과물이었습니다.



이후

개발자 → 문서 작성 → AI → 코드

사람은 무엇을 왜 만들지 문서로 확정하고,
AI가 코드를 생성합니다.
사람의 산출물이 코드에서 문서로 올라갑니다.

구현 비용과 판단 · 검증 비용

급락한 비용

"구현"

몇 주 걸리던 구현이 몇 분 만에 생성됩니다.
함수 하나를 짜는 일은 이제 싼 일입니다.

남은 비용

"판단과 검증"

무엇을 만들지 정하는 판단,
만들어진 것이 맞는지 보는 검증은
그대로 사람의 시간입니다.

최신 모델도 함수 수준 문제는 83%를 풀지만,
저장소 전체 수준의 과제에서는 16%에 그칩니다. 부분에서 유능하고, 전체에서 사람이 필요합니다.

사람의 일과 AI의 일

일	주담당	왜
방향과 우선순위 결정	사람	무엇이 중요한지는 업의 판단입니다
요구사항 확정	사람	틀린 목표를 주면 AI는 그것을 정확히 구현합니다
회의록 정리, 자료 분석	AI	패턴이 있는 정리는 AI가 빠르고 지치지 않습니다
코드 생성, 문서 초안	AI	생성은 싸졌습니다
아이디어 발상, 화면 설계	함께	AI가 넓히고, 사람이 고릅니다
최종 검증과 승인	사람	책임은 위임되지 않습니다

맡길 일의 판별 기준

내가 아는 일

"Exploitation"

각 단계의 수행 사항과 산출물,
그 평가 방식을 내가 확실히 알고 있는 일.
AI에게 시켜 볼 만합니다.
결과가 맞는지 내가 판정할 수 있기 때문입니다.

내가 잘 모르는 일

"Exploration"

내가 잘 알지 못하는 일.
AI의 보조를 받아 탐색해야 하고,
전체를 맡길 수는 없습니다.
결과가 좋아도 나빠도 판정할 수 없기 때문입니다.

※ 일 자체의 성질로도 같은 것을 물을 수 있습니다. 중간 판단이 필요한 일인가, 완료를 객관적으로 확인할 수 있는가.

개발 생명주기의 병목

폭포수 (waterfall)

한 방향, 한 번

요구사항 정의부터 배포까지 단계를 한 방향으로 밟습니다.

코드는 사람이 직접 씁니다.

구현 구간에 **전체 시간의 대부분**이 들어갔습니다.

애자일 (agile)

짧은 주기, 반복

회의와 피드백 주기를 1~2주로 줄였습니다.

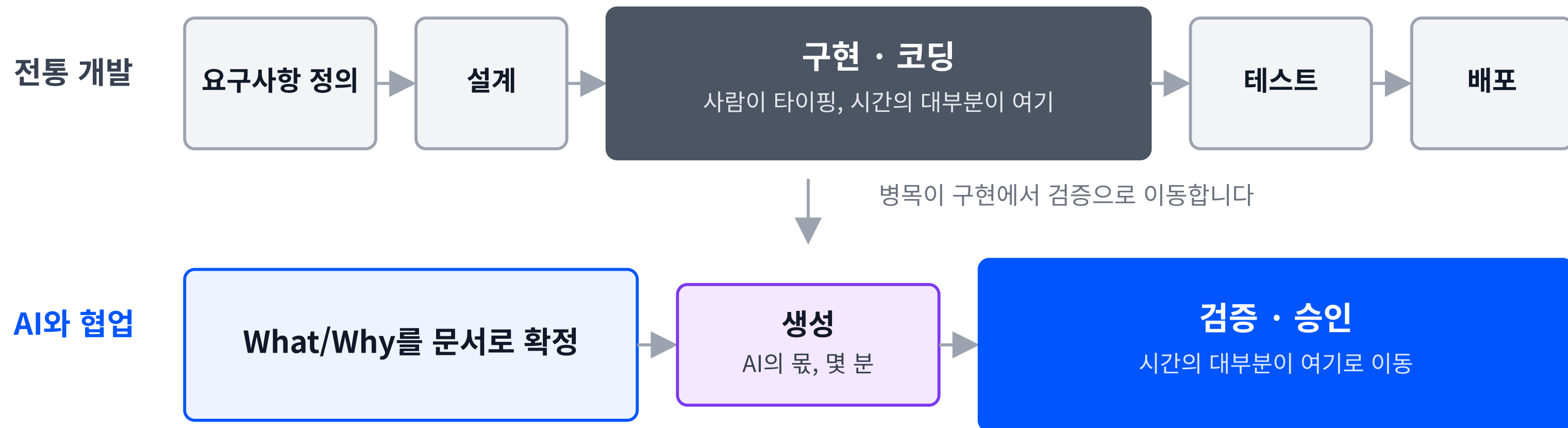
코드 생산은 여전히 사람의 수작업이었습니다.

요구가 바뀌는 속도를 **타이핑이 따라가지 못합니다.**

프로세스는 빨라졌는데 생산은 그대로인 상태.

이 병목이 풀리면 생명주기 전체가 다시 그려집니다.

AI-DLC의 단계 비중



병목은 이제 검증입니다. 그래서 문서와 판단이 사람의 일이 됩니다.

Vibe Coding과 명세 기반 개발

기준	감에 의존 (Vibe Coding)	명세로 통제
환경	휘발되는 채팅창	버전 관리되는 문서
입력	모호한 자연어 지시	명확한 제약과 규칙
결과	설명 못 하는 블랙박스	검증 가능한 산출물

- 프롬프트만 던지고 결과를 받는 방식은 2025년 무렵 Vibe Coding이라는 이름으로 유행했고, 곧 **검증 불가**라는 한계에 부딪혔습니다.
- 지금의 표준은 명세와 문서로 통제하는 개발입니다.

※ 주의: Vibe Coding은 신조어였다가 멸칭이 되었고, 지금은 쓰는 사람마다 뜻이 다릅니다. harness까지 갖춘 개발을 Vibe Coding이라 부르는 사람도 있으니, 들을 때 어떤 뜻으로 쓰는지 확인해야 합니다.

AI를 통제하는 다섯 계층

Prompt Engineering

무엇을 말할 것인가

Context Engineering

무엇을 알게 할 것인가

Harness Engineering

무엇을 쓰게 할 것인가

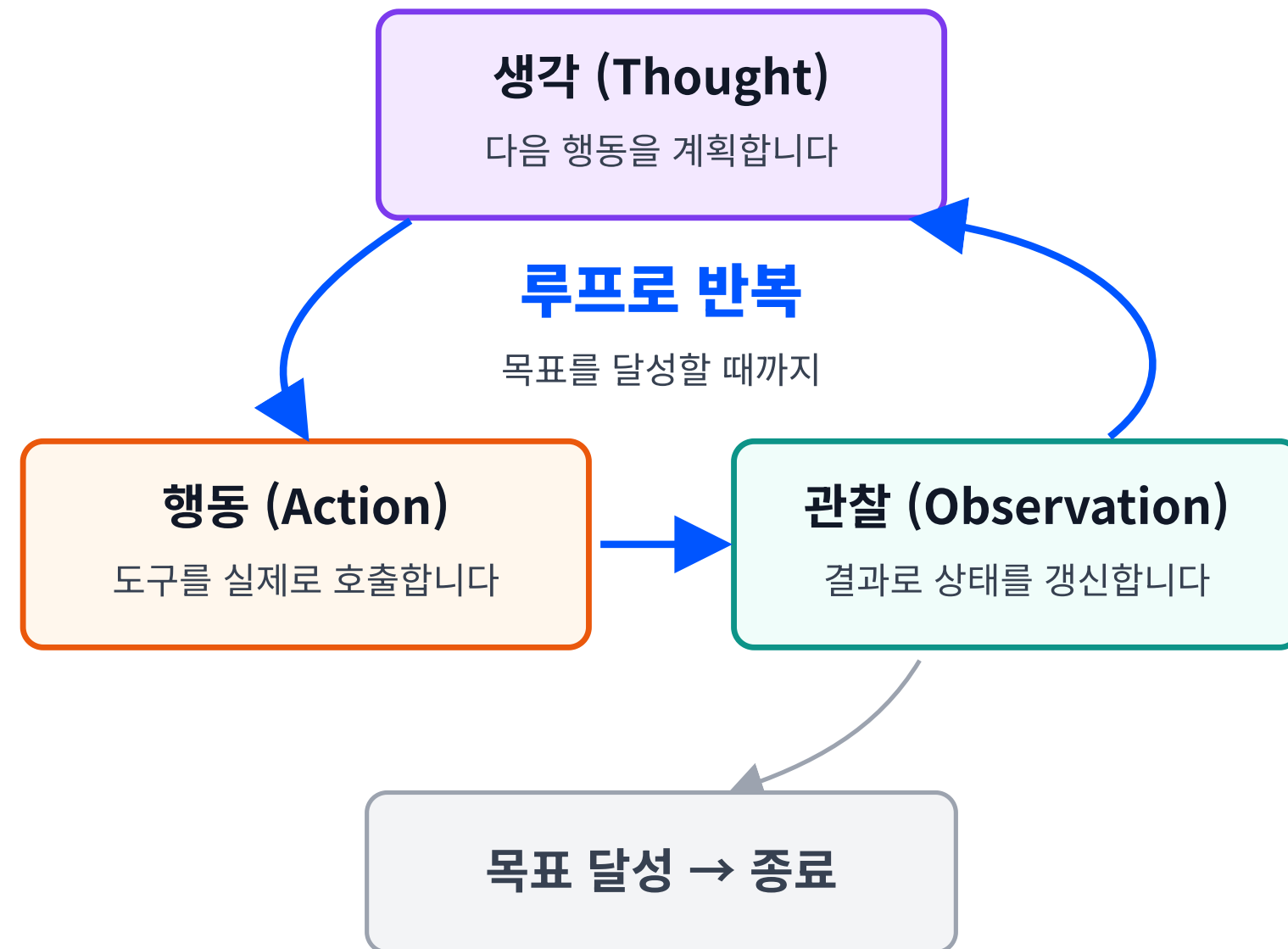
Loop Engineering

어떻게 반복할 것인가

Graph Engineering

어떻게 경로를 나눌 것인가

ReAct: AI가 문제를 푸는 흐름



- ReAct는 추론(Reasoning)과 행동(Acting)을 한 흐름에 묶은 문제 해결 방식입니다.
생각(Thought) · 행동(Action) · 관찰(Observation)이 **루프로 반복**되며 작업이 끝납니다.
- 에이전트가 "스스로 일한다"의 실체가 이 루프입니다.
루프를 읽을 줄 알면 AI가 왜 그렇게 움직였는지 보입니다.

작업 로그 읽기

[생각] 파일 전체를 읽기 전에, 함수 정의
 줄만 먼저 찾아 지도를 그리는 편이 빠르겠다.
[행동] > 검색 실행: `grep "def " day02_module.py`
[관찰] 함수 2개: `ask` (21행),
 `ask_endpoint` (38행, `POST /ask`에 연결).
[생각] 이름만으로는 설명을 못 쓴다.
 두 함수의 본문을 읽어야겠다.
[행동] > 파일 열기: `day02_module.py` 21~45행
[관찰] `ask`는 LLM 호출, `ask_endpoint`는
 그것을 HTTP 주소로 연결.
[생각] 이제 설명을 쓸 수 있다.
 정리해 파일로 저장하자.
[행동] > 파일 쓰기: `functions.md`
[관찰] 파일이 생성됐다. 완료 보고.

- 코딩 에이전트에게 작은 과제를 시키고
 실제 작업 과정을 그대로 옮긴 로그입니다.
- **생각**이 계획을 세우고, **행동**이 도구를 부르고,
 관찰이 다음 생각의 재료가 됩니다.
- 생각, 행동, 관찰이 세 차례 돌며 과제가
 끝났습니다.
 이 단위로 끊어 읽으면 AI가 무엇을 근거로
 다음 행동을 골랐는지 보입니다.

챕터 요약

- ✓ 구현 비용은 급락했고 판단과 검증의 비용은 남았습니다. 사람의 일은 코드에서 문서와 판단으로 옮겨 갑니다
- ✓ 기존 DLC와 AI-DLC의 차이: 구현은 AI의 몫이 되고, 문서 확정과 검증·승인이 사람의 몫으로 커집니다
- ✓ AI가 일하는 기본 루프: ReAct(Thought, Action, Observation)

AI-Native 개발 방법론

업의 분석과 워크플로우 설계

왜 문서인가 / 분해와 절차서 / 요구사항과 워크플로우

학습 목표

- 자기 업(또는 처음 보는 업)을 목적, 입력, 산출물, 판단 기준으로 분해할 수 있습니다.
- 업무를 태스크 단위로 쪼개 순서와 흐름을 정하는 절차서(SoP)를 만들 수 있습니다.
- 절차서를 AI가 읽기 쉽게 다듬어 Agentic Workflow 설계도로 만들 수 있습니다.

✓ AI와 협업하는 개발

▸ 업의 분석과 워크플로우 설계

컨텍스트 엔지니어링과 구조화 출력

업무 지시



사수

자, 회의 참가는 처음이지? 내용들 잘 기록했어?

그럼 회의록으로 '잘' 정리해서 공유해줘.

네



신입

...네!

무슨 일이 벌어질까요?

예상 시나리오 1: 좋은 의미로 신입이 왜 신입인지 알게 됩니다.

예상 시나리오 2: 나쁜 의미로 신입이 왜 신입인지 알게 됩니다.

일을 시키는 능력과 글쓰기

- 첫 질문은 "어떻게 잘 쓸까"가 아니라 **"누구를 위해 쓰는가"**입니다.
- 무엇을 말하려는가, 왜 그런가, 어떻게 할 것인가. 이 순서만 지켜도 **구조 잡힌 문서**가 됩니다.
- AI가 초안을 써 주는 시대에 남는 역량은 좋은 글과 나쁜 글을 **구별하고 고치는 눈**입니다.

시킬 수 없는 지시

사수의 말 그대로

"회의록으로 '잘' 정리해서 공유해 줘."

누가 읽는지, 무엇이 '잘'인지, 언제까지인지가 없습니다. 빠진 자리는 받는 쪽이 추측으로 채웁니다.

시킬 수 있는 지시

같은 일을 다시 적으면

"불참자를 포함한 팀원 전원이 결정사항과 할 일을 월요일 전에 알 수 있도록, 회의 녹음을 {결정사항 / 할 일 / 미결} 세 칸으로 정리해 금요일 18시 전에 팀 채널에 올려 주세요."

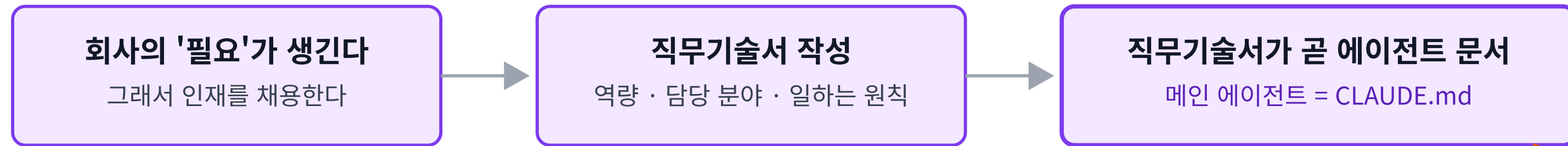
누구를 위해, 무엇을, 왜, 무엇이 확인되면 끝인지가 문장 안에 다 있습니다.

문서화 능력이 **AI 시대의 핵심 역량**입니다.

머릿속에서 처리하던 일을 적어낸 문서가, AI에게 공급할 수 있는 유일한 형태의 지식입니다.

일을 넘기는 두 줄기

역할의 길 — 누가 말하는가



절차의 길 — 무엇을 어떻게 하는가



업 분석의 네 가지 질문



- 어떤 업이든 **목적**, **입력**, **산출물**, **판단 기준**의 네 질문으로 분해할 수 있습니다.
- 특정 직무 전용 틀이 아니라 어느 업에나 대는 **범용 프레임**입니다.
- 판단 기준**은 "이럴 땐 어떻게 정하지"에 해당합니다.
머릿속에만 있으면 문서로 넘어가지 못합니다.

SoP 작성법: 여섯 단계

1 리스트업과 조사

알고 있는 내용을 전부 적고,
부족한 부분은 자료나
규정을 조사합니다

2 목적과 범위 (Objective, Scope)

이 작업이 왜 있는지 밝히고
전체 작업 흐름을
구성합니다

3 수행 절차 (Action)

각 작업 단계에서
무엇을 하는지
기술합니다

4 입력과 산출물 규격

각 단계가 받는 것과
내놓는 것의 규격을
잡습니다

5 판단 방침 (Policy)

각 단계에서 무엇을 보고
정하는지, 무엇을
조심하는지 정리합니다

6 이슈 대응 방식

알려진 이슈와 예상하지
못한 이슈에 어떻게
대응할지 정리합니다

SoP는 남이 이 문서만 보고 같은 품질로 일할 수 있게 적은 절차입니다.

암묵지에서 절차서로

암묵지 (Before)

"매주 금요일 회의 끝나면 내가 회의록 정리해서 공유해야 해. 녹음 다시 듣고 중요한 것만 추려서 메일 보내면 돼."

본인은 할 수 있습니다.
그러나 이 상태로는 아무에게도 시킬 수 없습니다.



절차서 (After)

목적과 입력, 산출물이 서고
단계마다 판단 기준이 붙고
이슈 대응까지 적힌 문서

머릿속에만 있던 것이 글자로 나왔습니다.
이 문서는 신입에게도, AI에게도 넘길 수 있습니다.

완성본: 주간 회의록 요약 · 공유 SoP

SoP: 주간 회의록 요약 · 공유

목적

불참자를 포함한 전원이 결정사항과 할 일을 월요일 전에 안다.

입력 · 산출물

입력: 회의 녹음(또는 메모), 지난 회의록

산출물: 회의록 문서 1건, 공유 메일 1통

절차

1. 녹음을 들으며 발언을 시간순으로 메모한다.
2. 메모를 {결정사항 / 할 일 / 미결} 세 칸으로 나눈다.
판단 기준: "누가 언제까지"가 붙는 문장이 할 일이다.

절차 (계속)

3. 할 일마다 담당자와 기한을 붙인다.
판단 기준: 회의에서 지정이 없었으면 [담당 미정]으로 적는다. 지어내지 않는다.
4. 지난 회의의 할 일과 대조해 완료와 이월을 표시한다.
5. 정리본을 팀 채널에 올리고 링크를 메일로 보낸다.

완료 기준

금요일 18시 전 발송 확인.

이슈 대응

녹음이 없으면 참석자 2인에게 메모를 받아 2번부터 진행한다.

처음 보는 업 분석법: 다섯 단계

1 자료로 파악

문서와 실제 자료부터
봅니다

2 가설 제작

목적, 입력, 산출물,
판단 기준으로
가설을 세웁니다

3 질문 변환

가설표의 빈 칸을
질문 목록으로
바꿉니다

4 채움

문거나 조사해
칸을 채웁니다

5 절차 초안

절차를 세워
확인받습니다

가설표의 빈 칸은 질문으로 바꿔 채웁니다.
네 칸이 다 차야 절차 초안을 세울 수 있습니다.

예시: 쇼핑몰 반품 처리 분석

질문	자료로 세운 가설
목적	반품 요청을 규정대로 처리해 환불까지 끝낸다
입력	반품 신청서, 도착한 반품 상품
산출물	환불 처리 완료, 고객 안내 메일
판단 기준	(비어 있음)

- 자료만으로는 판단 기준 칸이 비었습니다. 이 빈 칸을 **질문으로 변환**합니다.
- "반품 도착 후 환불 여부는 누가, 무엇을 보고 판정합니까?"
- 담당자에게 묻거나, 규정 문서를 찾아 채웁니다. 채워지면 절차 초안을 세워 확인받습니다.

문제, 과제, 대책의 구분

가설을 세우기 전에 문제·과제·대책의 층을 먼저 나눕니다.

1 문제(상태)

"이상과 현실의 갭"

반품이 지난달보다 두 배로 늘었다.

2 과제(방향)

"갭을 메우기 위해 할 일"

반품 사유를 유형별로 파악한다.

3 대책(실행)

"구체 행동"

포장 규격 문서를 만들어 배포한다.

층을 섞으면 분석이 무너집니다.

"무엇이 문제인가"를 묻는 자리에 대책을 답하지 않습니다.

MECE: 가설 제작 품질 검사

- MECE(Mutually Exclusive, Collectively Exhaustive)는 **겹침 없이, 빠짐 없이** 나누라는 분류 규율입니다.
- 빠짐이 있으면 틀린 결론으로 가고, 겹침이 있으면 같은 일을 두 번 합니다.
- 분해가 끝났다고 선언하기 전에 겹침과 빠짐을 한 번 검사합니다.
- **가설 제작**의 품질을 검사하는 도구입니다.

겹치고 빠진 분해

반품 처리를 이렇게 나눴다면

신청 접수 / 상품 확인 / 고객 응대
환불 실행이 빠졌고, 고객 응대는 모든 단계에서 하는 일입니다.

겹침 없이, 빠짐 없이

같은 업을 다시 나눈다면

신청 접수 / 상품 확인 / 환불 실행 / 결과 통보
넷을 합치면 빠짐과 겹침이 없습니다.

5 Whys: 근본 원인 추적

- 왜를 다섯 번 반복해 표면의 증상에서 근본 원인까지 내려가는 기법이 **5 Whys**입니다.
- 다섯 번째 질문에 이르면 사람이 아니라 **구조와 문서의 부재**가 원인으로 나옵니다. 그래서 답도 문서로 돌아옵니다.

한 번 묻고 끝내지 않습니다.
답을 받으면 그 답에 다시 왜를 묻습니다.

반품이 늘었다

- 왜? 포장 파손이 많다
- 왜? 포장 규격이 상품별로 제각각이다
- 왜? 규격을 정한 문서가 없다
- 왜? 포장 담당이 바뀔 때마다 구두로만 전달됐다
- 왜? 절차를 적는 규칙 자체가 없었다.

업 분석 도구 상자

도구	하는 일	언제 꺼내나
로직트리 (Logic Tree)	문제를 갈래로 쪼개 원인과 대책을 정리합니다	큰 문제를 작게 나눌 때
피시본 다이어그램 (Fishbone)	원인을 사람 · 방법 · 재료 등 갈래별로 펼칩니다	원인을 넓게 훑을 때
MECE	겹침과 빠짐을 검사합니다	분해를 끝냈다고 선언하기 전에
5 Whys	왜를 반복해 근본 원인까지 내려갑니다	표면 증상만 보일 때
피라미드 원칙 (Pyramid Principle)	결론을 먼저 두고 근거를 아래로 배열합니다	분석 결과를 전달할 때
OODA 루프	관찰 · 방향 설정 · 결정 · 행동을 반복합니다	상황이 계속 바뀔 때

요구사항과 제약조건 적는 법

항목	적는 것	예
요구	무엇을 해 달라는 것인가	회의록을 요약한다
목적	그것으로 달성되는 상태	불참자를 포함한 전원이 결정사항을 안다
사용자	산출물을 쓰는 사람	팀원 전원, 팀장
입출력 예시	실제 입력 1건과 기대 출력 1건	회의 메모 → 세 칸 요약문
도구 · 제약	쓸 수 있는 것, 쓰면 안 되는 것	사내 메신저만, 외부 전송 금지
완료 기준	무엇이 확인되면 끝인가	금요일 18시 전 발송 확인

분해 렌즈와 일감 문서

네 가지 질문

"업을 쪼개는 렌즈"

목적, 입력, 산출물, 판단 기준.
일이 무엇인지 알아내려고 씁니다.
산출물은 분해표입니다.

여섯 항목 서식

"일을 넘기는 서식"

요구, 목적, 사용자, 입출력 예시,
도구와 제약, 완료 기준.
알아낸 일을 시키려고 적습니다.
산출물은 일감 문서입니다.

질문으로 분해하고, 서식으로 넘깁니다. 분석 문서와 발주 문서는 다릅니다.

정성적인 지시 → 결정론적인 명세서

영역	정성적 지시	결정론적 명세
성능	검색을 "빠르게" 개선해 줘	1,000건 조회 시 응답을 200ms 이내로 보장할 것
보안	데이터를 "안전하게" 보호해 줘	모든 접근에 토큰 인증을 적용하고 평문 저장을 금지할 것
품질	"깔끔하게" 작성해 줘	함수당 한 가지 일만 하게 하고 검사 도구를 통과할 것

※ "빠르게, 안전하게, 깔끔하게"는 사람끼리도 다르게 읽는 말입니다. 숫자와 규칙으로 바꿔야 검증할 수 있습니다.

문서에 들어가야만 하는 세 가지 요소

기능의 목적과 범위

무엇을

무엇을 만드는지, 입출력이 무엇인지 적습니다.
구현 방법은 여기 적지 않습니다.

제약과 규칙

어떻게

이 프로젝트에서는 어떤 방식으로 만드는지.
기술 선택과 금지 사항이 들어갑니다.

진행 단계와 체크리스트

재현

어떤 순서로 진행하는지.
누가 해도 같은 순서가 되도록 적습니다.

워크플로우 설계



- 절차서(SoP)가 서면, 태스크와 순서는 이미 그 안에 있습니다.
- 남은 일은 절차를 **AI가 읽기 쉽게 다듬는** 것입니다.
단계마다 입력과 산출물 규격, 완료 조건을 붙여 조사, 분석, 작성처럼 단계별로 넘길 수 있게 합니다.
- 그렇게 다듬으면 **Agentic Workflow 설계도**, 혹은 **스킬 문서**가 됩니다.

역할 문서의 다섯 항목

항목	적는 것
이름 · 역할	무엇을 담당하는가
입력 · 출력	무엇을 받아 무엇을 내는가
도구	무엇을 쓸 수 있는가
제약	하면 안 되는 것
실패 시 위험	잘못되면 무슨 일이 생기는가

※ 사람에게 같은 항목을 적으면 직무기술서(Job Description)입니다.

역할 문서의 유형

절차서를 받아 수행하는 에이전트

"입력과 출력이 정해진다"

표준이 잡히는 작업이라 다섯 항목을 전부 적을 수 있습니다. 실험실의 테크니션처럼, SoP를 받아 수행하는 자리입니다.

목적만 있고 방법이 없는 주체

"태도와 원칙이 핵심이다"

무엇이 나올지 미리 정할 수 없어 입출력을 표준화할 수 없습니다. 어떤 원리와 원칙으로 판단할지를 적습니다.

실습합시다

일을 문서로 적는 순간, 그 문서가
AI에게 공급할 컨텍스트(Context)가 됩니다.

챕터 요약

- ✓ 설명하지 못하는 일은 시킬 수 없습니다. 문서로 적어야 사람에게도 AI에게도 넘길 수 있습니다
- ✓ 업은 목적, 입력, 산출물, 판단 기준의 네 질문으로 분해하고, 분해의 품질은 MECE로, 근본 원인은 5 Whys로 검사합니다
- ✓ SoP는 여섯 단계로 씁니다. 리스트업과 조사, 목적과 범위, 수행 절차, 입력과 산출물 규격, 판단 방침, 이슈 대응
- ✓ 처음 보는 업은 자료로 가설을 세우고, 가설표에 남은 빈 칸을 질문으로 바꿔 채웁니다
- ✓ 요구사항은 요구, 목적, 사용자, 입출력 예시, 도구와 제약, 완료 기준의 여섯 항목으로 적고, 정성적인 표현은 숫자와 규칙으로 바꿉니다
- ✓ 업무를 태스크 단위로 쪼개 순서와 흐름을 정하면 절차서가 되고, 절차서를 AI가 읽기 쉽게 다듬으면 Agentic Workflow 설계도가 됩니다
- ✓ 역할 문서는 두 유형에 따라 쓰는 방식이 다릅니다. 입출력이 표준화되는 상대에게는 다섯 항목을 전부 적고, 목적만 있고 방법이 없는 상대에게는 태도와 원칙을 적습니다
- ✓ 일을 문서로 적는 순간, 그 문서가 AI에게 공급할 컨텍스트가 됩니다

AI-Native 개발 방법론

컨텍스트 엔지니어링과 구조화 출력

편집술 / 구획과 예산 / 구조화 출력

학습 목표

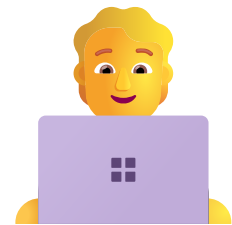
- 같은 모델이라도 공급한 컨텍스트에 따라 결과가 달라지는 구조를, Context Window의 구획과 한계를 들어 설명할 수 있습니다.
- 규격을 정하지 않고 받은 응답이 프로그램에서 깨지는 지점을, 실행 결과를 들어 짚을 수 있습니다.
- 출력을 JSON Schema로 강제해 객체로 받고, 담당자별 메시지 생성까지 이어지는 코드를 작성할 수 있습니다.

✓ AI와 협업하는 개발

✓ 업의 분석과 워크플로우 설계

▶ **컨텍스트 엔지니어링과 구조화 출력**

사라진 결정의 근거

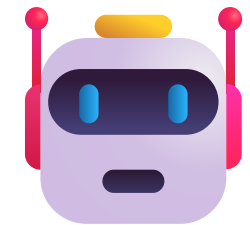


개발자

(석 달 전) 인증 만료는 15분으로 하죠.

(지금) 근데 왜 15분이었지?

완료했습니다!

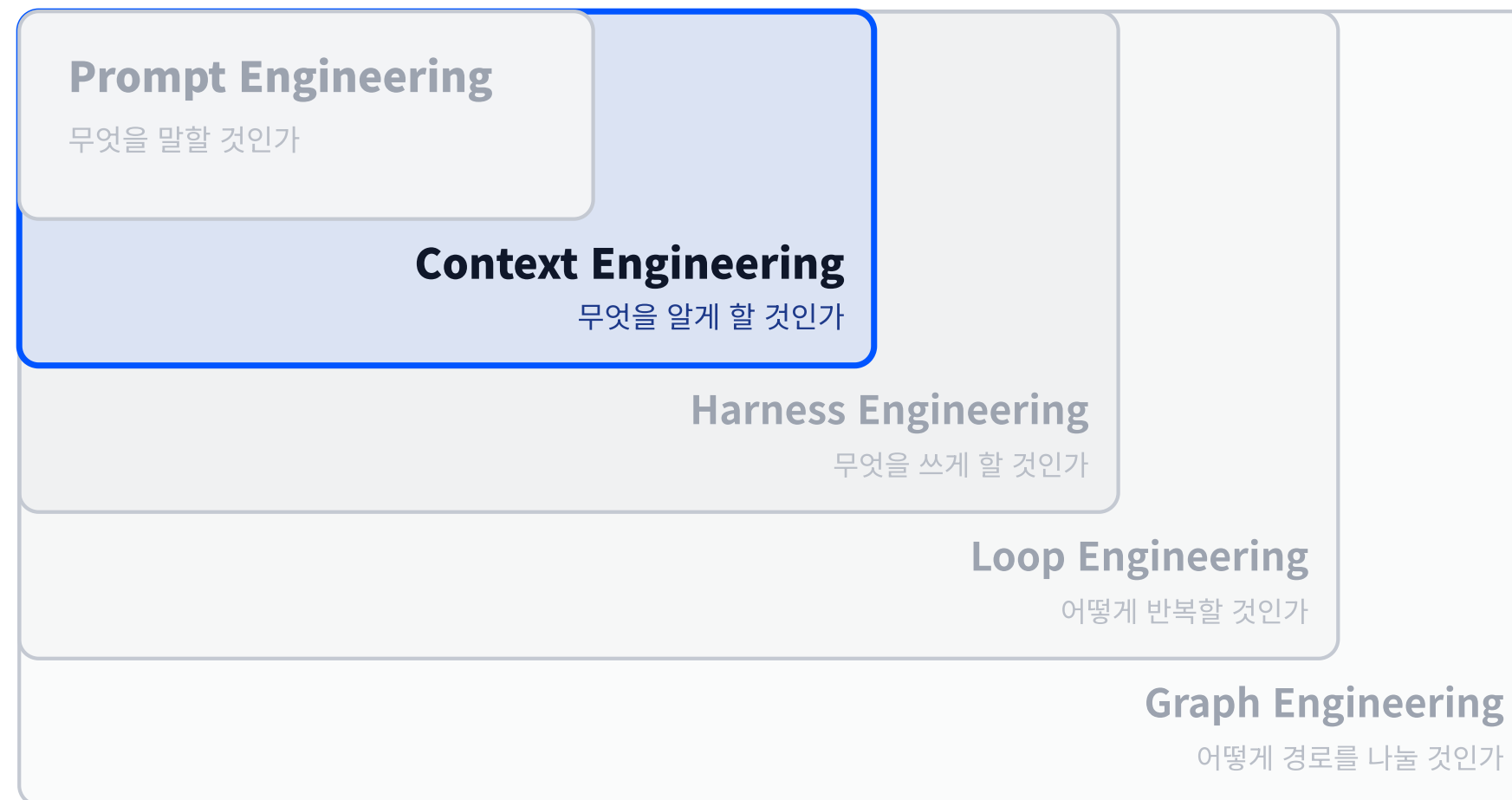


코딩 에이전트

죄송합니다, 그 결정의 기록을 찾을 수 없습니다.

대화는 창에서 밀려나면 사라집니다. 결정의 이유가 문서로 남지 않으면, AI는 같은 질문에 추측으로 답합니다.

프롬프트 엔지니어링과 컨텍스트 엔지니어링



- 한 번의 지시문을 다듬는 기술이 **프롬프트 엔지니어링(Prompt Engineering)**, 모델이 **무엇을 알게 할 것인가**를 설계하는 기술이 **컨텍스트 엔지니어링(Context Engineering)**입니다.
- 지시가 좋아도 재료가 나쁘면 결과가 나쁩니다.
지시가 부실해도 재료가 좋으면 결과는 크게 나쁘지 않습니다.
- 잘 다듬은 지시문은 템플릿으로 저장해 다시 쓰고,
컨텍스트 문서도 **자산**으로 남습니다.

AI의 품질은 모델이 아니라 내가 공급한 **컨텍스트**가 결정합니다.

입력은 문서로 다듬고, 출력은 스키마로 받습니다.

Context Window의 두 가지 한계

기억 상실

"Context Overflow"

내용이 길어질수록 앞에서 정한 원칙과 제약을 잊습니다.

Context Window에서 밀려난 것은 없는 것과 같습니다.

중간 유실

"Lost in the Middle"

자료가 방대하면 한가운데 있는 핵심 조건을 무시하는 경향이 있습니다. 넣었다고 읽히는 것이 아닙니다.

많이 넣을수록 좋은 것이 아니라, 중요한 것이 묻힙니다.

컨텍스트 엔지니어링의 절반은 덜어내는 일입니다.

채우는 기술이 아니라 고르는 기술입니다. 중요한 것을 중요하게 남기고, 불필요한 것을 걷어냅니다.

Context Window의 네 구획

Context Window, 한정된 지면



- Context Window는 한정된 지면입니다. 구획마다 **남기는 규칙**이 다릅니다.
- 고정 지침은 항상 남기고, 근거 자료는 지금 질문에 필요한 조각만 넣습니다.
- 자리가 모자라면 **덜 중요한 것부터 덜어냅니다.** 오래되었다는 이유가 아니라, 지금 덜 중요하다는 이유로 뺍니다.
- 사고 여백은 반드시 먼저 확보합니다. 여백이 없으면 답변 품질이 떨어집니다.

Context Window를 쓰는 두 규칙

위치

"앞과 뒤가 잘 읽힌다"

긴 자료는 앞에 두고, 지금 시키는 지시와 질문은 맨 뒤에 둡니다. 가운데에 묻힌 내용은 모델이 놓칩니다.

꼭 채우지 않기

"채울수록 정확도가 떨어진다"

입력이 길어질수록 정확도는 내려갑니다. 최대 크기까지 채우지 않고, 사고 여백을 먼저 남깁니다.

문서 자산이 있을 때의 지시문

매번 처음부터 (Before)

"로그인 화면 만들어 줘. 스타일은 기존 화면이랑 맞춰 주고, 버튼은 파란색 계열, 입력칸은 이메일이랑 비밀번호 두 개. 이메일 형식 검사하고 비밀번호는 8자 이상, 실패하면 입력칸 아래 빨간 글씨로 메시지. 아 그리고 에러는 전부 공통 에러 클래스 쓰기로 했었지? 함수는 너무 길어지지 않게..."

매번 길게 설명하고, 매번 결과가 다릅니다.
설명이 빠진 자리는 추측으로 채워집니다.



문서를 참조시켜 (After)

"규칙 문서와 절차를 읽고,
로그인 화면을 만들어 줘."

지시는 짧아지고 결과는 일정해집니다.
과거의 결정이 문서로 살아 있기 때문입니다.

규격 없는 요약 지시

```
from litellm import completion

INSTRUCTION = (
    "다음 주간 회의 메모를 요약해 주세요. "
    "결정된 것, 해야 할 일(누가·언제까지), "
    "아직 결론이 나지 않은 것을 정리해 주세요."
)

def summarize_freeform(memo):
    prompt = INSTRUCTION + "\n\n" + memo
    res = completion(
        model="openai/gpt-5.6-luna",
        messages=[{"role": "user",
                    "content": prompt}])
    return res.choices[0].message.content
```

입력한 회의 메모

주간 회의 메모 (금요일)

- 다음 배포는 수요일로 확정했다.
- 접속 오류 재현 조건을 **민수 씨가**
다음 회의 전까지 정리하기로 했다.
- 매뉴얼 개편은 다다음 주에 다시 논의한다.
- 신규 문의 응대 템플릿 초안은
지현 씨가 화요일까지 만들기로 했다.
- 서버 증설 예산은 결론을 내지 못했다.

- 지시문은 한 번 써 두고 계속 씁니다. 회의 때마다 새로 들어오는 것은 **메모 한 덩어리**뿐입니다.
- **담당자와 기한**은 문장 속에 섞여 있고, 받을 답의 모양은 아무것도 정하지 않았습니다.

규격 없이 받은 응답

결정된 사항

- 다음 배포는 **수요일에 진행**한다.
- 매뉴얼 개편은 **다다음 주에 다시 논의**한다.

해야 할 일

담당자	할 일	기한
-----	-----	----

---	---	---
-----	-----	-----

민수 씨	접속 오류 재현 조건 정리	다음 회의 전까지
------	----------------	-----------

지현 씨	신규 문의 응대 템플릿 초안 작성	화요일까지
------	--------------------	-------

아직 결론이 나지 않은 사항

- **서버 증설 예산**은 결론을 내리지 못했으며, 추가 논의가 필요하다.

- 사람이 읽기에는 좋은 답입니다. 그러나 **담당자와 기한**은 표 한 칸 안의 글자일 뿐이라, 프로그램이 꺼내 쓸 자리가 아닙니다.

자유 텍스트의 파싱 실패

그 응답을 파싱하는 코드

```
import json

text = summarize_freeform(MEETING_MEMO)
data = json.loads(text)
```

```
Traceback (most recent call last):
  File "summarize.py", line 12, in <module>
    data = json.loads(text)
  json.decoder.JSONDecodeError: Expecting
value: line 1 column 1 (char 0)
```

- 사람에게 친절한 답이 프로그램에는 **쓸 수 없는 답**입니다. 첫 글자가 여는 중괄호가 아니면 파싱은 그 자리에서 멈춥니다.
- "JSON으로 답해 줘"라고 적어 두어도 인사말이나 설명 문장이 한 줄 붙으면 그대로 깨집니다.
- 이 실패는 **시그럽습니다**. 어디서 멈췄는지 곧바로 알려 줍니다.

같은 지시, 두 번의 응답

```
=== 자유 호출 1회차 ===
## 결정된 사항
- 다음 배포는 **수요일에 진행**한다.
- 매뉴얼 개편은 **다다음 주에 다시 논의**한다.

## 아직 결론이 나지 않은 사항
- **서버 증설 예산**은 결론을 내리지
  못했으며, 추가 논의가 필요하다.

=== 자유 호출 2회차 ===
## 결정된 사항
- 다음 배포: **수요일 진행**
- 매뉴얼 개편: **다다음 주에 재논의**

## 아직 결론이 나지 않은 사항
- 서버 증설 예산: **결정 보류**
```

```
import re

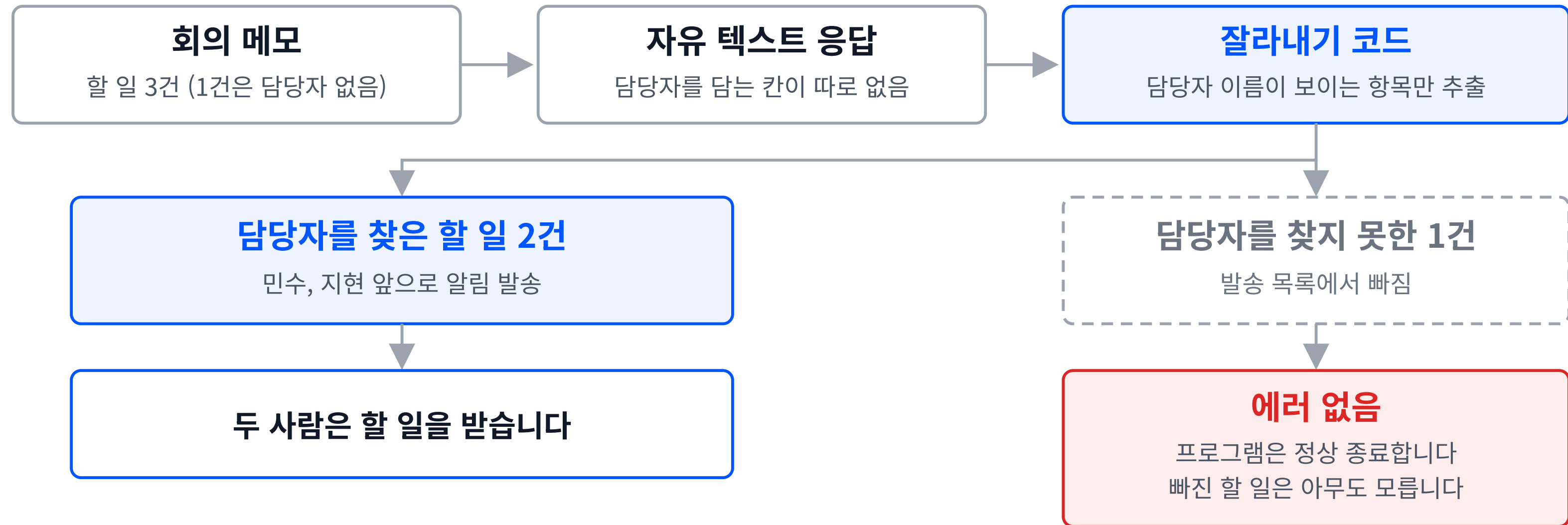
PAT = r"- \*\*(.+?)\*\*"

def pick_open_issue(text):
    body = text.split("## 아직")[1]
    m = re.search(PAT, body)
    return m.group(1) if m else None

# 1회차 → '서버 증설 예산'
# 2회차 → None
```

- **굵은 글씨의 자리**가 옮겨 가자, 첫 응답에 맞춰 쓴 잘라내기 코드가 아무것도 찾지 못합니다.

에러가 없음 != 문제가 없음



출력의 모양 선언

```
from pydantic import BaseModel

class ActionItem(BaseModel):
    task: str          # 할 일
    owner: str         # 담당자
    due: str | None    # 기한 (없으면 None)

class MeetingSummary(BaseModel):
    decisions: list[str]
    action_items: list[ActionItem]
    open_questions: list[str]
```

- 출력의 필드 이름과 타입을 선언하면, 그 모양이 **JSON Schema로 변환되어** 모델에 전달됩니다.
- 작업을 지시받은 모델은 그 규격에 맞는 답만 내고, 어긋난 답은 **검증에서 걸립니다.**
- 모양을 정하는 일은 **무엇을 남기고 무엇을 버릴지 정하는 일**입니다. 선언에 없는 내용은 답에 담기지 않고, 없을 수 있는 값은 `str | None`으로 적어 두어야 "없음"을 적을 자리가 생깁니다.

규격을 건 호출과 수신 JSON

```
SYSTEM = ("회의 메모에서 결정사항, 할 일, "  
          "미결 사항을 추출한다. 담당자가 "  
          "없는 할 일의 owner는 '미정'으로 "  
          "적는다. 메모에 없는 내용을 "  
          "지어내지 않는다.")
```

```
res = completion(  
    model="openai/gpt-5.6-luna",  
    messages=[{"role": "system",  
               "content": SYSTEM},  
              {"role": "user",  
               "content": MEETING_MEMO}],  
    response_format=MeetingSummary)
```

```
{  
  "decisions": [  
    "다음 배포는 수요일로 확정했다.",  
    "매뉴얼 개편은 다다음 주에 다시 논의한다."  
  ],  
  "action_items": [  
    {"task": "접속 오류 재현 조건 정리",  
      "owner": "민수",  
      "due": "다음 회의 전까지"},  
    {"task": "신규 문의 응대 템플릿 초안 작성",  
      "owner": "지현",  
      "due": "화요일까지"}  
  ],  
  "open_questions": [  
    "서버 증설 예산은 결론을 내지 못했다."]  
}
```

파싱 없이 객체로 접근

```
raw = res.choices[0].message.content
summary = MeetingSummary.model_validate_json(raw)

summary.decisions[0]          # '다음 배포는 수요일로 확정했다.'
summary.action_items[0].owner # '민수'
summary.action_items[0].due    # '다음 회의 전까지'
len(summary.action_items)      # 2
```

- 문자열을 자르거나 정규식을 쓸 일이 없습니다. 필드 이름으로 곧바로 꺼냅니다.
- 선언에 없는 필드를 꺼내면 그 자리에서 에러가 납니다. 값이 조용히 사라지지 않습니다.

담당자별 메시지 생성

```
GREETING = ("{} 님, 이번 회의에서 "
            "정해진 할 일입니다.")

def notify_owners(summary):
    out = {}
    for it in summary.action_items:
        due = ""
        if it.due:
            due = f" (기한: {it.due})"
        head = GREETING.format(it.owner)
        line = f"\n- {it.task}{due}"
        prev = out.get(it.owner, head)
        out[it.owner] = prev + line
    return out
```

```
msgs = notify_owners(summary)
for owner, msg in msgs.items():
    print(f"[{owner}]")
    print(msg)
```

[민수]

민수 님, 이번 회의에서 정해진 할 일입니다.

- 접속 오류 재현 조건 정리 (기한: 다음 회의 전까지)

[지현]

지현 님, 이번 회의에서 정해진 할 일입니다.

- 신규 문의 응대 템플릿 초안 작성 (기한: 화요일까지)

모양 검사와 값 검사

인풋

```
{ "task": "접속 오류 재현 조건 정리",  
  "owner":            ← 비어 있음  
  "due": "2026-13-45" }           ← 달력에 없는 날짜
```

값 검증 없는 모양 검사기의 결과

```
1 validation error for ActionItemShapeOnly  
owner  
Field required
```

값 검증기 코드

```
@field_validator("due")  
@classmethod  
def due_must_exist_on_calendar(cls, v):  
    if v and re.fullmatch(  
        r"\d{4}-\d{2}-\d{2}", v):  
        date.fromisoformat(v)  
    return v
```

```
2 validation errors for ActionItem  
owner  
Field required  
due  
Value error, month must be in 1..12
```

품질 게이트

관문	검사하는 것
기능 정합	요구한 것이 전부 구현됐는가, 없는 기능이 멋대로 붙지 않았는가
테스트	새 코드에 테스트가 있는가, 전부 통과하는가
코드 품질	비효율과 중복, 규칙 위반
보안	노출된 비밀 값, 위험한 입력 처리
구조 정합	기존 시스템과 어긋나지 않는가

회의록 요약 일감에 적어 둔 검증 방법

요구사항 서식 · 완료 기준
금요일 18시 전 발송 확인

스키마 검증 · 통과 조건
ValidationError 0건

- 네 관문은 각각 다른 것을 봅니다.
하나를 통과해도 다른 관문이 막을 수 있습니다.
- 다 만든 뒤에 오류를 찾는 대신,
기준 미달 산출물은 합쳐지기 전에 막습니다.
- 검증 방법이 적혀 있어야
끝났다는 보고를 믿을 수 있습니다.

Google, What to look for in a code review (eng-practices) · Sonar, Introduction to quality gates (SonarQube Docs) · GitHub, About protected branches / 2026-08-23 확인

AI-Native 개발 방법론 / 컨텍스트 엔지니어링과 구조화 출력 / 구조화 출력

자주 나오는 질문

규격을 강제하지 않으면 왜 안 되나

파싱이 그 자리에서 깨집니다. 형식은 호출마다 달라질 수 있어, 문자열을 잘라 쓰는 코드가 다음 호출에서 멈춥니다. 빠진 값은 에러 없이 지나갑니다.

메모에 정보가 없으면 에러가 나나

아닙니다. 필수로 선언한 칸은 반드시 채워지고, 근거가 없으면 모델이 추론하거나 지어냅니다. 없을 때 무엇을 적을지 지시문에 정해 두거나, `str | None`으로 선언해 "없음"을 적을 자리를 만듭니다.

매번 바뀌는 것은 무엇인가

회의 메모뿐입니다. 작업 지시와 출력 규격은 고정입니다. 그래서 지시문과 스키마가 문서 자산으로 남습니다.

규격을 강제하면 검증은 필요 없나

필요합니다. 강제가 보증하는 것은 모양뿐입니다. 값이 옳은지는 검증기를 따로 붙여 봅니다.

실습합시다

챕터 요약

- ✓ AI의 품질은 모델이 아니라 공급한 컨텍스트가 결정합니다
- ✓ Context Window는 한정된 지면이라 덜 중요한 것부터 덜어냅니다
- ✓ 규격 없이 받은 응답은 파싱에서 깨지고, 빠진 값은 에러 없이 지나갑니다
- ✓ 출력을 JSON Schema로 강제하면 프로그램이 그대로 받아 다음 단계로 넘깁니다
- ✓ 요구사항의 완료 기준과 스키마 검증을 적어 두는 것이 품질 게이트의 최소 형태입니다

